¿Por qué el algoritmo de Dijkstra es Greedy?

En este resumen no vamos a repasar al algoritmo. Lo asumimos super conocido.

La pregunta que nos hacemos en clase es "¿por qué este algoritmo es Greedy?". Esto puede aplicar también a los algoritmos de Prim y Kruskal.

La respuesta nunca puede ser "porque realiza una acción greedy que nos lleva a un óptimo local, esperando que nos lleve a un óptimo global". Ese texto de memoria aplicable a cualquier cosa no indica entender qué es un algoritmo greedy. Tampoco puede ser "porque uso un heap". Usar un heap u ordenar no hace al algoritmo greedy. Incluso, podríamos no usar un heap, podríamos simplemente guardar en un arreglo/lista e iterarla por completo buscando el mínimo. ¿Ineficiente? Si, pero no cambia la esencia del algoritmo (si, por supuesto, su complejidad).

Si, en efecto tenemos que centrarnos en la regla greedy. Si no la podemos enunciar, el algoritmo probablemente no sea greedy. Luego tenemos que poder enunciar cuál es el óptimo local al que nos lleva esa regla greedy. Si no podemos enunciar el óptimo local (local a este momento, considerando las circunstancias actuales), entonces seguramente el algoritmo no sea greedy, o hay algo que estamos pensando mal. Un ejemplo importante a revisar es el del ejercicio resuelto de la guía de algoritmos greedy. Revisar ese ejemplo, porque se mencionan un par de errores que incluso van directo a un caso dónde el algoritmo planteado no sería greedy.

La regla greedy y el óptimo local

En cada iteración vamos a aplicar como regla obtener el vértice más cercano (que no haya sido ya analizado). Es decir, de lo que resta, el vértice más cercano. Esta regla es la que nos permite que, si nosotros estábamos buscando el camino mínimo hacia un vértice destino particular, si ahora estamos viendo dicho vértice, es imposible mejorar su distancia (con aristas con pesos no negativos), por lo que podemos parar de iterar.

¿Cuál es el óptimo local al que nos lleva esto? Primero, ya podemos dar por terminado el análisis de un vértice (es decir, por la imposibilidad antes mencionada, hay un vértice menos que analizar). A su vez, si desde este vértice (imposible de mejorar) encontramos un camino hacia sus adyacentes con mejor peso, entonces lo mejoramos y guardamos este resultado (nos acercamos más al óptimo, partiendo de esta información de esta iteración actual). El próximo vértice que saquemos (sea uno de estos mejorados, u otro) es imposible de mejorar, por lo que nos acercamos cada vez más a tener lo mejor para cada vértice.

Notar que en ningún lado estoy haciendo mención de exactamente cómo guardamos esta información, porque eso no hace a la esencia de "lo greedy". Es importante, por supuesto, pero no es lo que analizamos aquí.